

Software Requirements Specification

IITM PRAVARTHAK TECHNOLOGIES

GROUP 7-ENHANCEMENT TO PSQLCOPY

AMALENDU MANOJ – 11

FATHIMA MAHIM – 35

KRISHNADEV P V - 44

Table of Contents

Table of Contents	2
1. Introduction	2
1.1 Purpose	2
1.2 Definitions, Acronyms, and Abbreviations	2
1.3 Product Scope	3
1.4 References	3
2. Overall Description	3
2.1 Existing issues in plsqlcopy	3
2.2 Proposed Enhancements	3
• Parallel Copy Operations	4
• Batch Processing	4
• Add batch processing for splitting large data files into smaller chunks for more manageable copy operations.	4
• Extended File Format Support	4
• Preview Data	4
• Pre-Validation	4
2.3 Design and Implementation Constraints	4
• Backward compatibility with existing COPY functionality.	4
• Efficient memory usage and minimal performance overhead.	4
• Ensuring transactional integrity when using batch operations.	4
3. Specific Requirements	4
3.1 Functional Requirements	4
3.1.1 Improved Resource Usage	5
3.1.2 Improved User Interface	7
3.1.3 Security Features	7
3.1.4 Extended File Format Support	1
3.1.5 Error Handling & Recovery	1
3.2 Non-Functional Requirements	1
• Scalability	1
• Performance.	1
• Reliability.	1
• Usability.	1
3.3 Flow chart	1
i. Usecase Diagram	1
ii . How copy command format checking works :-	1
4. Conclusion	1

1. Introduction

1.1 Purpose

The purpose of this document is to outline the requirements for enhancing PostgreSQL's COPY function with additional capabilities such as data preview, pre-validation, parallel copy operations, batch processing, and extended support for multiple file formats. These improvements aim to enhance data import/export efficiency, improve usability, and support modern data workflows.

1.2 Definitions, Acronyms, and Abbreviations

- **COPY:** A command in PostgreSQL for importing/exporting large volumes of data.
- **PostgreSQL:** An open-source relational database management system.
- **JSON/JSONL:** Lightweight data formats used for structured data representation.
- **Avro:** Columnar storage format optimized for big data applications.

1.3 Product Scope

The enhancements will extend PostgreSQL's COPY command to:

- *Allow users to preview data before execution.*
- *Perform pre-validation to ensure data integrity before insertion.*
- *Enable parallel processing for improved performance.*
- *Implement batch processing to handle large datasets efficiently.*
- *Support additional file formats including JSON, JSONL and Avro.*

1.4 References

- PostgreSQL Documentation: <https://www.postgresql.org/docs/>
- <https://github.com/postgres/postgres/blob/master/src/bin/psql/copy>.

2. Overall Description

2.1 Existing issues in `psqlcopy`

- *COPY currently lacks data preview capabilities.*
- *Pre-validation checks must be implemented manually by users.*
- *Single-threaded execution limits performance on large datasets.*
- *Lack of built-in support for modern file formats such as Json and Avro.*

2.2 Proposed Enhancements

- **Parallel Copy Operations**
 - *Implement support for parallel COPY operations to utilize multiple CPU cores for faster data transfer.*
 - *Allow users to specify the number of parallel threads or workers.*
- **Batch Processing**
 - **Add batch processing for splitting large data files into smaller chunks for more manageable copy operations.**
- **Extended File Format Support**
 - *Support additional data formats: JSON, JSONL and Avro.*

- **Preview Data**

- *Allow previewing the first few rows of the data file before performing the COPY operation.*

- **Pre-Validation**

- *Add a validation mode to check file compatibility, column mapping, and data integrity without executing the COPY command.*

2.3 Design and Implementation Constraints

- Backward compatibility with existing COPY functionality.
- Efficient memory usage and minimal performance overhead.
- Ensuring transactional integrity when using batch operations.

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Improved Resource Usage

- *Implement multi-threaded execution for parallel copy operations.*
- *Optimize memory usage for handling large data files.*
- *Introduce an intelligent resource scheduler to dynamically allocate resources based on system load.*

3.1.2 Improved User Interface

- *Introduce an interactive preview mode for data before execution.*
- *Provide detailed error messages and validation reports.*

○

3.1.3 Security Features

- *Ensure role-based access control (RBAC) for bulk data imports.*
- *Implement logging and auditing for **COPY** operations.*
- *Encrypt data transfers for enhanced security.*

3.1.4 Extended File Format Support

- *Implement native support for JSON, JSONL and Avro file formats.*
- *Provide an extensible framework to allow users to define custom data formats.*

3.1.5 Error Handling & Recovery

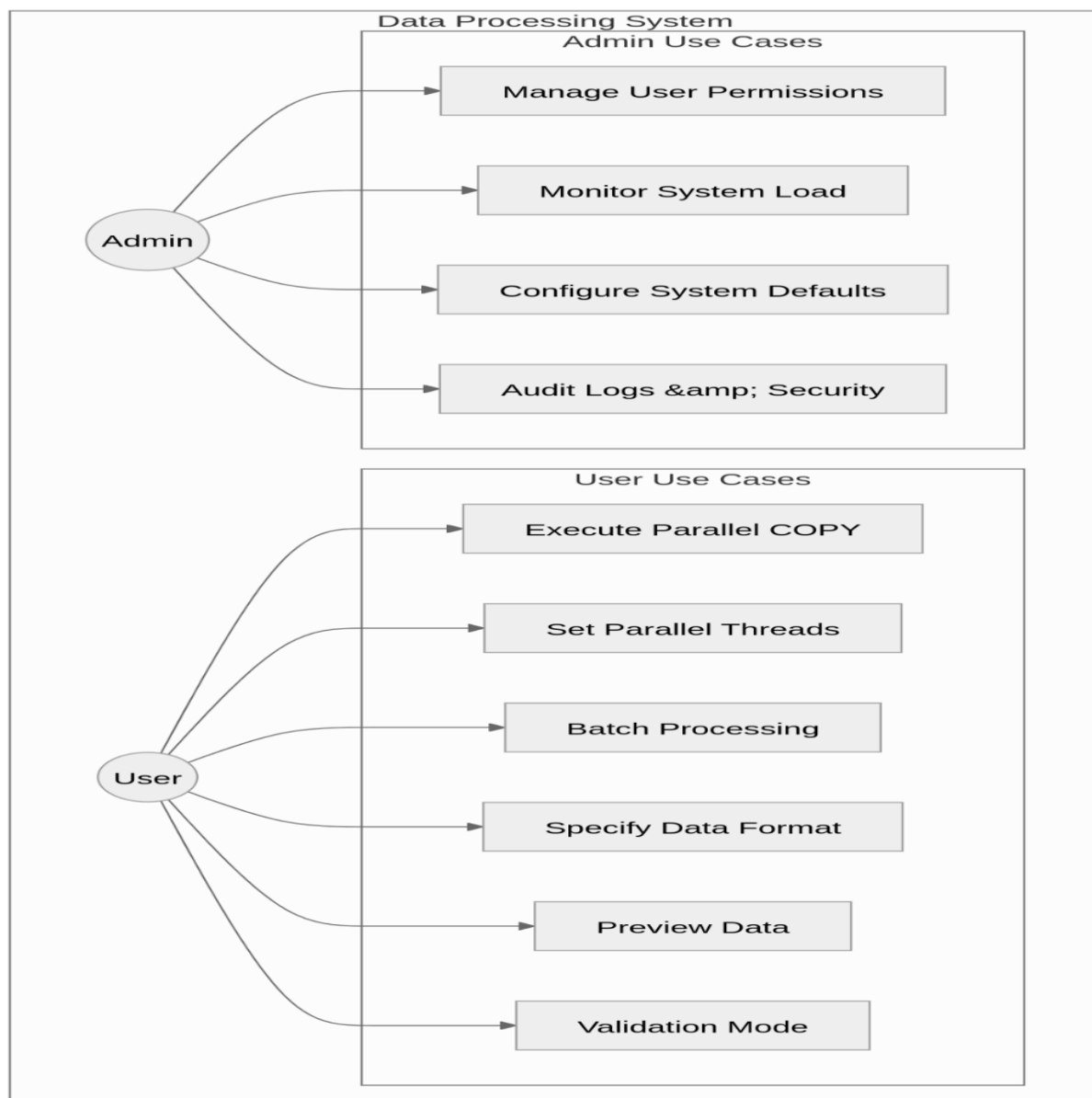
- *Implement automatic rollback on failure to maintain data integrity.*
- *Provide retry mechanisms for failed operations.*
- *Generate detailed logs for debugging and diagnostics.*

3.2 Non-Functional Requirements

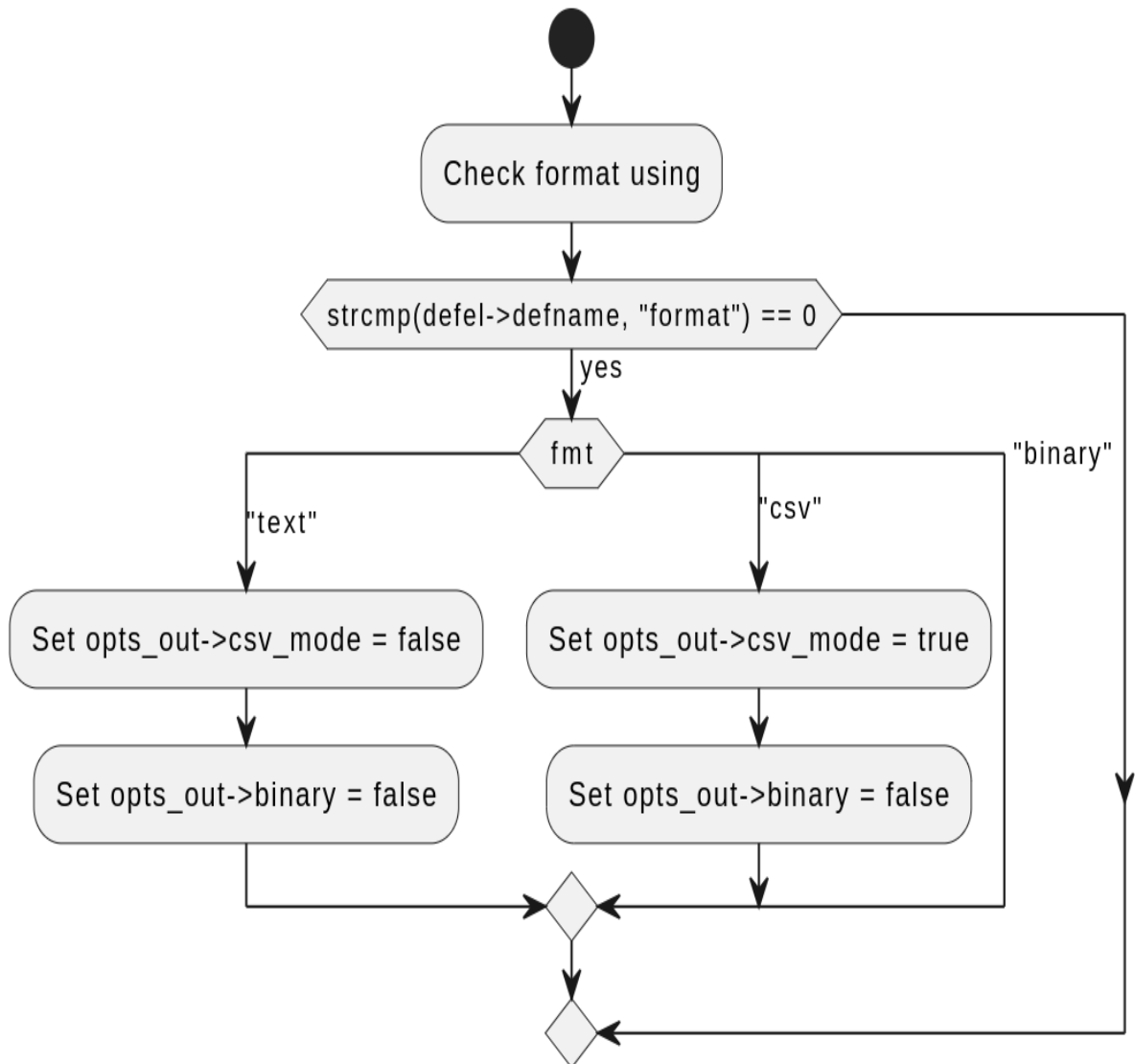
- **Scalability:** *Support high-throughput data ingestion for enterprise-level applications.*
- **Performance:** *Reduce data import time by at least 50% for large datasets.*
- **Reliability:** *Ensure robust error handling and rollback mechanisms.*
- **Usability:** *Provide clear documentation and user guidance for new features.*

3.3 Flow chart

i. Usecase Diagram



ii . How copy command format checking works :-



4. Conclusion

The proposed enhancements to PostgreSQL's COPY function will significantly improve data import/export efficiency, enable parallel and batch processing, and introduce support for modern file formats. These improvements will make PostgreSQL more suitable for large-scale data workloads and analytical processing.